Module 4: SQL & Power BI for Data Analytics

Comprehensive Guide for Machine Learning and AI Professionals

Introduction

SQL (Structured Query Language) and **Power BI (Business Intelligence)** are two pillars of modern data analytics. SQL gives you the power to extract, clean, and transform data from databases—often containing billions of records. Power BI transforms that data into compelling visualizations and interactive dashboards that drive business decisions.

This module equips you with industry-grade skills that will make you invaluable in any data-driven organization. Whether you're querying a corporate database or building executive dashboards, these tools are non-negotiable in a data professional's toolkit. The combination of SQL for data preparation and Power BI for visualization creates a complete analytics pipeline—from raw data to actionable insights.

Part 1: SQL for Data Science

Introduction to Databases

A **database** is an organized collection of structured data stored in tables (rows and columns), optimized for efficient retrieval and analysis. Modern databases use the **relational model**, where tables are related through keys.

Why SQL for Data Science?

- Extract specific data from massive datasets (billions of rows)
- Clean, transform, and prepare data for modeling
- Perform complex aggregations and analyses
- Enable reproducible data pipelines
- Work independently without depending on engineers

Basics of SQL: DML, DDL, DCL, and Data Types

SQL commands are categorized into three main types:

1. DDL (Data Definition Language) - Defines database structure

- `CREATE` - Create tables and databases
- `ALTER` - Modify table structure
- `DROP` - Delete tables and databases

2. DML (Data Manipulation Language) - Manipulates data

- `SELECT` - Retrieve data
- `INSERT` - Add new records
- `UPDATE` - Modify existing records
- `DELETE` - Remove records

3. DCL (Data Control Language) - Controls access

- `GRANT` - Give permissions
- `REVOKE` - Remove permissions

Common Data Types:

Data Type	Description	Example
INT	Integer numbers	42, -100
FLOAT/DECIMAL	Decimal numbers	3.14, 99.99
VARCHAR(n)	Text up to n characters	'John', 'Data Science'
CHAR(n)	Fixed-length text	'CA', 'NY'
DATE	Calendar date	'2024-11-27'
DATETIME/TIMESTAMP	Date and time	'2024-11-27 14:30:00'
BOOLEAN	True/False	TRUE, FALSE

Common SQL Commands

SELECT, FROM, WHERE:

```
-- Retrieve columns from a table
SELECT column1, column2, column3
FROM table_name;

-- Filter with WHERE clause
SELECT name, salary, department
FROM employees
WHERE department = 'Sales' AND salary &gt; 50000;

-- Get all columns
SELECT *
FROM customers;

-- Limit results
SELECT * FROM orders LIMIT 10;  -- First 10 rows
```

Aliases for readability:

```
SELECT
    e.employee_id,
    e.first_name,
    e.last_name,
    d.department_name
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id;
```

Logical Operators in SQL

```
-- AND: All conditions must be true
SELECT * FROM products
WHERE category = 'Electronics' AND price &gt; 500;

-- OR: At least one condition must be true
SELECT * FROM orders
WHERE status = 'Completed' OR status = 'Pending';

-- NOT: Reverses condition
SELECT * FROM customers
WHERE NOT city = 'New York';

-- Combining operators
SELECT * FROM sales
WHERE (region = 'North' OR region = 'South')
AND year = 2024;
```

Filtering and Sorting

Advanced Filtering:

```
-- IN operator: Check if value is in list
SELECT * FROM employees
WHERE department IN ('Sales', 'Marketing', 'IT');

-- NOT IN
SELECT * FROM products
WHERE category NOT IN ('Discontinued', 'Inactive');

-- BETWEEN: Range filtering
SELECT * FROM orders
WHERE order_date BETWEEN '2024-01-01' AND '2024-12-31';

-- LIKE: Pattern matching
SELECT * FROM customers
WHERE email LIKE '%@gmail.com';  -- Ends with @gmail.com
SELECT * FROM customers
WHERE first_name LIKE 'J%';      -- Starts with J

-- IS NULL / IS NOT NULL
SELECT * FROM products WHERE description IS NULL;
```

Sorting with ORDER BY:

```
-- Single column ascending (default)
SELECT * FROM employees
ORDER BY salary;

-- Descending order
SELECT * FROM employees
ORDER BY salary DESC;

-- Multiple columns
SELECT * FROM orders
ORDER BY customer_id ASC, order_date DESC;
```

Grouping with GROUP BY:

```
-- Group sales by department
SELECT department, COUNT(*) as employee_count
FROM employees
GROUP BY department;

-- HAVING filters grouped results
SELECT department, AVG(salary) as avg_salary
FROM employees
GROUP BY department
HAVING AVG(salary) &gt; 50000;
```

SQL Joins

Joins combine data from multiple tables based on relationships.

INNER JOIN - Returns only matching records:

```
SELECT
    o.order_id,
    c.customer_name,
    o.order_date
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id;
```

LEFT JOIN - All from left table + matching from right:

```
SELECT
    c.customer_name,
    COUNT(o.order_id) as total_orders
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id;
```

RIGHT JOIN - All from right table + matching from left:

```
SELECT *
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id;
```

FULL OUTER JOIN - All records from both tables:

```
SELECT *
FROM employees e
FULL OUTER JOIN departments d ON e.dept_id = d.dept_id;
```

SQL Aggregations

Common aggregation functions:

```
-- COUNT: Number of rows
SELECT COUNT(*) as total_employees FROM employees;
SELECT COUNT(DISTINCT department) as dept_count FROM employees;

-- SUM: Total of values
SELECT SUM(salary) as total_payroll FROM employees;

-- AVG: Average value
SELECT AVG(salary) as avg_salary FROM employees;

-- MIN/MAX: Minimum and maximum
SELECT
    MIN(salary) as lowest_salary,
    MAX(salary) as highest_salary
FROM employees;

-- Multiple aggregations
SELECT
    department,
    COUNT(*) as emp_count,
    AVG(salary) as avg_salary,
    MAX(salary) as max_salary
FROM employees
GROUP BY department;
```

CASE Function - Conditional logic:

```
SELECT
    employee_id,
    name,
    salary,
    CASE
        WHEN salary &lt; 40000 THEN 'Junior'
        WHEN salary &lt; 60000 THEN 'Senior'
        ELSE 'Lead'
```

```
    END as salary_band
FROM employees;
```

DATE Functions:

```
-- Extract parts of dates
SELECT
    order_id,
    order_date,
    YEAR(order_date) as year,
    MONTH(order_date) as month,
    DAY(order_date) as day
FROM orders;

-- Date arithmetic
SELECT
    order_id,
    order_date,
    DATEADD(DAY, 7, order_date) as week_later
FROM orders;

-- Current date
SELECT GETDATE() as today;  -- SQL Server
SELECT NOW() as today;      -- MySQL
```

NULL Handling:

```
-- COALESCE: Return first non-null value
SELECT
    product_id,
    COALESCE(description, 'No description') as desc
FROM products;

-- ISNULL (SQL Server)
SELECT
    customer_id,
    ISNULL(phone, 'No phone') as contact
FROM customers;
```

Subqueries and Temporary Tables

Subqueries execute multiple queries together:

```
-- Subquery in WHERE clause
SELECT * FROM employees
WHERE salary &gt; (SELECT AVG(salary) FROM employees);

-- Subquery in FROM clause
SELECT dept, avg_sal
FROM (
    SELECT department as dept, AVG(salary) as avg_sal
    FROM employees
```

```
    GROUP BY department
) sub_result
WHERE avg_sal &gt; 50000;

-- Subquery in SELECT
SELECT
    department,
    (SELECT COUNT(*) FROM employees e2
     WHERE e2.dept_id = e1.dept_id) as emp_count
FROM employees e1;
```

Temporary Tables (CTEs - Common Table Expressions):

```
-- WITH clause for readability
WITH dept_averages AS (
    SELECT
        department,
        AVG(salary) as avg_salary
    FROM employees
    GROUP BY department
)
SELECT
    e.name,
    e.salary,
    da.avg_salary
FROM employees e
JOIN dept_averages da ON e.department = da.department
WHERE e.salary &gt; da.avg_salary;
```

Window Functions

Window functions perform calculations across rows without collapsing results:

```
-- ROW_NUMBER: Unique sequential number
SELECT
    name,
    salary,
    ROW_NUMBER() OVER (ORDER BY salary DESC) as rank
FROM employees;

-- RANK: Rank with ties
SELECT
    name,
    salary,
    RANK() OVER (ORDER BY salary DESC) as rank
FROM employees;

-- DENSE_RANK: Rank without gaps
SELECT
    name,
    salary,
    DENSE_RANK() OVER (ORDER BY salary DESC) as rank
FROM employees;
```

```sql
-- LAG/LEAD: Access previous/next row
SELECT
    month,
    revenue,
    LAG(revenue) OVER (ORDER BY month) as prev_month,
    LEAD(revenue) OVER (ORDER BY month) as next_month
FROM monthly_sales;

-- Running totals
SELECT
    order_date,
    amount,
    SUM(amount) OVER (ORDER BY order_date) as running_total
FROM orders;

-- Partition calculations
SELECT
    department,
    name,
    salary,
    AVG(salary) OVER (PARTITION BY department) as dept_avg
FROM employees;
```

Part 2: Introduction to Power BI

What is Business Intelligence?

Business Intelligence (BI) is the process of converting raw data into actionable insights. It encompasses:

- Data collection and integration
- Data analysis and visualization
- Interactive dashboards and reports
- Data-driven decision making

Power BI Introduction

Power BI is Microsoft's analytics platform that connects to data, visualizes it, and shares insights.
Components:

- **Power BI Desktop** - Create reports and visualizations
- **Power BI Service** - Cloud-based sharing and dashboards
- **Power BI Mobile** - Mobile access to dashboards
- **Power Query** - Data transformation
- **Power Pivot** - Data modeling
- **DAX** - Data Analysis Expressions (formulas)

Power BI Workflow

1. **Connect** - Import data from various sources
2. **Transform** - Clean and reshape data using Power Query
3. **Model** - Create relationships and hierarchies
4. **Analyze** - Create measures and calculations with DAX
5. **Visualize** - Build interactive reports and dashboards
6. **Share** - Publish to Power BI Service

Data Import and Visualizations

Data Import Options:

```
- Excel files
- CSV files
- SQL databases
- Web data
- APIs (REST)
- Cloud services (Azure, Salesforce, Google Analytics)
- SharePoint lists
- Power Query for web scraping
```

Why Visualization?

- Humans process visuals 60,000x faster than text
- Patterns become immediately apparent
- Communicate insights effectively
- Enable interactive exploration

Visualization Types:

Chart Type	Best For	Example
Line Chart	Trends over time	Sales by month
Bar Chart	Comparing categories	Revenue by region
Pie Chart	Part-to-whole relationships	Market share
Scatter Plot	Correlations	Price vs. quantity
Map	Geographic data	Sales by country
KPI Card	Key metrics	Total revenue
Gauge	Performance vs. target	Completion %
Matrix	Crosstab analysis	Sales by region and product

Categorical Data Visualization:

```
Best charts: Bar chart, Column chart, Pie chart, Donut chart
Example: Product categories, customer segments, regions
```

Trend Data Visualization:

```
Best charts: Line chart, Area chart, Combo chart
Example: Sales over time, website traffic, stock prices
```

Slicers - Interactive filtering:

```
- Dropdown slicer - Select single or multiple values
- List slicer - Scrollable list of values
- Buttons - Toggle between views
```

Formatting Visuals:

- Colors and themes
- Data labels and legends
- Axis formatting
- Hover tooltips
- Conditional formatting

Part 3: Power Query - Data Transformation

Power Query Introduction

Power Query (integrated in Power BI) enables data transformation without leaving the interface.
Benefits:

- Clean messy data
- Combine multiple sources
- Create feature engineering
- Standardize formats
- Remove duplicates

Data Transformation Steps

1. **Load Data** - Connect to source
2. **Remove unnecessary columns** - Keep only relevant data
3. **Handle missing values** - Fill or remove
4. **Clean text** - Trim, uppercase, remove special characters
5. **Create new columns** - Feature engineering

6. **Filter rows** - Remove invalid records
7. **Merge/Combine** - Join multiple tables

Power Query Transformations

Column Transformations:

```
- Split column by delimiter
- Merge columns
- Change data type
- Replace values
- Extract text
- Format as percentage
- Create conditional column
```

Row Transformations:

```
- Remove duplicates
- Remove errors
- Filter by value
- Sort data
- Keep/remove rows
```

Combine Queries:

```
- Merge (SQL JOIN equivalent)
- Append (Stack tables vertically)
- Union (Combine similar queries)
```

Part 4: Power Pivot and DAX

Power Pivot - Data Modeling

Power Pivot enables creating relationships between tables and calculations.

Key Concepts:

- **Relationships** - Connect tables via keys
- **Cardinality** - One-to-many, many-to-many
- **Relationships View** - Visual diagram of connections

Calculated Columns vs Measures

Calculated Columns:

- Added to table as new column
- Calculated row-by-row
- Takes up storage
- Example: Age from birth date

```
Age = YEAR(TODAY()) - YEAR([BirthDate])
```

Measures:

- Aggregated calculations
- Calculated on-demand
- No storage overhead
- Example: Total sales

```
Total Sales = SUM([Sales Amount])
```

DAX Introduction and Syntax

DAX (Data Analysis Expressions) - Formulas for calculations in Power BI.

Basic Syntax:

```
MeasureName = Expression

-- Example
Total Revenue = SUM([Amount])
Average Price = AVERAGE([Price])
```

Common DAX Functions:

```
-- Aggregation functions
SUM([Amount])
AVERAGE([Price])
COUNT([ProductID])
DISTINCTCOUNT([CustomerID])
MIN([Date])
MAX([Quantity])

-- Logical functions
IF([Sales] &gt; 1000, "High", "Low")

-- Text functions
UPPER([Name])
LEN([Email])
```

```
CONCATENATE([FirstName], " ", [LastName])

-- Date functions
YEAR([OrderDate])
MONTH([OrderDate])
TODAY()
DATE(2024, 12, 25)
```

Filter Functions:

```
-- CALCULATE with filters
Total West Sales = CALCULATE(
    SUM([Sales]),
    [Region] = "West"
)

-- ALL: Remove filters
Total All Sales = CALCULATE(
    SUM([Sales]),
    ALL([Date])
)

-- FILTER: Apply conditions
High Value Sales = CALCULATE(
    SUM([Sales]),
    FILTER(
        Sales,
        Sales[Amount] &gt; 1000
    )
)
```

Time Intelligence Functions:

```
-- Year-to-date
YTD Sales = CALCULATE(
    SUM([Sales]),
    DATESYTD([Date])
)

-- Previous month
Prior Month Sales = CALCULATE(
    SUM([Sales]),
    PREVIOUSMONTH([Date])
)

-- Same period last year
Last Year Sales = CALCULATE(
    SUM([Sales]),
    SAMEPERIODLASTYEAR([Date])
)
```

Part 5: Dashboards and Advanced Features

Power BI Service - Publishing and Sharing

1. **Create Dashboard** - Combine multiple visualizations
2. **Pin Visuals** - Add from reports to dashboard
3. **Add Content** - Text boxes, images, web content
4. **Share Dashboard** - Publish to workspace or share with users
5. **Row-Level Security (RLS)** - Restrict data by user role

Dashboard Elements:

```
- Visuals (charts, maps, KPIs)
- Slicers (filters)
- Text boxes (descriptions, titles)
- Images (logos, backgrounds)
- Web content (embedded links)
- Videos
- Buttons (for interactivity)
```

Advanced Features

Visual Interactions:

- Click on one visual to filter others
- Drill-through to detailed pages
- Drill-down within hierarchies

Conditional Formatting:

- Color scale based on values
- Data bar visualization
- Icon sets
- Custom rules

Creating Buttons:

- Navigate between pages
- Back buttons
- Action buttons with bookmarks

Python Script Visuals:

- Import Python-based visualizations
- Custom analytics from Python libraries
- Advanced statistical plots

SQL Best Practices for Data Scientists

1. **SELECT only needed columns** - Improves performance
2. **Filter early with WHERE** - Reduce data before joins
3. **Use indexes on filter/join columns** - Faster retrieval
4. **Use meaningful table aliases** - Improves readability
5. **Comment your code** - Enable collaboration
6. **Test with LIMIT first** - Check logic before full run
7. **Use CTEs for complex logic** - More readable than nested queries
8. **Avoid SELECT *** - Specify columns explicitly
9. **Store queries in version control** - Track changes
10. **Document your data model** - Enable others to understand relationships

References

1. Chodorow, K. (2024). SQL and Relational Database Theory: Who Killed My Database? O'Reilly Media.
2. Schultze, R. (2024). T-SQL Fundamentals (4th ed.). Microsoft Press.
3. Ferrari, A., & Russo, M. (2023). The Definitive Guide to DAX (2nd ed.). Microsoft Press.
4. Kolaczyk, E., & Csárdi, G. (2024). Statistical Analysis of Network Data with R. Springer.
5. Microsoft. (2024). Power BI Documentation. Retrieved from https://docs.microsoft.com/power-bi/
6. Real Python. (2024). SQL Tutorial. Retrieved from https://realpython.com/
7. DataCamp. (2024). SQL for Data Science. Retrieved from https://www.datacamp.com/
8. Mode Analytics. (2024). SQL Tutorial. Retrieved from https://mode.com/sql-tutorial/
9. Kimball, R., & Ross, M. (2023). The Data Warehouse Toolkit (4th ed.). Wiley.
10. GeeksforGeeks. (2024). SQL for Data Science. Retrieved from https://www.geeksforgeeks.org/
[1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12] [13] [14] [15] [16] [17] [18] [19] [20]

⁂

1. https://towardsdatascience.com/all-the-sql-a-data-scientist-needs-to-know-7a5328176a67/
2. https://www.irjmets.com/uploadedfiles/paper/issue_3_march_2024/51366/final/fin_irjmets1712297274.pdf
3. https://statisticsbyjim.com/regression/multicollinearity-in-regression-analysis/
4. https://www.datacamp.com/tutorial/multicollinearity
5. https://www.geeksforgeeks.org/machine-learning/multicollinearity-in-regression-analysis/
6. https://www.ijrti.org/papers/IJRTI2407036.pdf
7. https://www.reddit.com/r/learnmachinelearning/comments/1coh5u0/in_linear_regression_no_multicollinearity/
8. https://www.kdnuggets.com/2022/10/classification-metrics-walkthrough-logistic-regression-accuracy-precision-recall-roc.html
9. https://www.geeksforgeeks.org/machine-learning/random-forest-algorithm-in-machine-learning/

10. https://ieeexplore.ieee.org/document/10441559/
11. https://www.graphpad.com/guides/prism/latest/curve-fitting/reg_logistic_roc_curves.htm
12. https://www.geeksforgeeks.org/data-science/sql-for-data-science/
13. https://keylabs.ai/blog/random-forest-ensemble-learning-technique/
14. https://www.upgrad.com/blog/sql-for-data-science/
15. https://www.coursera.org/learn/sql-for-data-science
16. https://pub.towardsai.net/sql-best-practices-every-data-scientist-and-analyst-should-know-bcc03598ed6d
17. https://learn.microsoft.com/en-us/power-bi/transform-model/desktop-quickstart-learn-dax-basics
18. https://www.journalijar.com/article/47394/classifying-the-supervised-machine-learning-and-comparing-the-performances-of-the-algorithms/
19. https://www.sprinkledata.com/blogs/top-10-practices-for-writing-sql-queries
20. https://www.datacamp.com/tutorial/power-bi-dax-tutorial-for-beginners